

Sessions

MSFconsole can manage multiple modules at the same time. This is one of the many reasons it provides the user with so much flexibility. This is done with the use of **Sessions**, which creates dedicated control interfaces for all of your deployed modules.

Once several sessions are created, we can switch between them and link a different module to one of the backgrounded sessions to run on it or turn them into jobs. Note that once a session is placed in the background, it will continue to run, and our connection to the target host will persist. Sessions can, however, die if something goes wrong during the payload runtime, causing the communication channel to tear down.

Using Sessions

While running any available exploits or auxiliary modules in msfconsole, we can background the session as long as they form a channel of communication with the target host. This can be done either by pressing the **[CTRL] + [Z]** key combination or by typing the **background** command in the case of Meterpreter stages. This will prompt us with a confirmation message. After accepting the prompt, we will be taken back to the msfconsole prompt (**msf6 >**) and will immediately be able to launch a different module.

Listing Active Sessions

We can use the **sessions** command to view our currently active sessions.

Sessions				
msf6 exploit(windows/smb/psexec_psh) > sessions				
Active sessions				
=====				
Id	Name	Type	Information	Connection
--	----	----	-----	-----
1		meterpreter	x86/windows NT AUTHORITY\SYSTEM @ MS01	10.10.10.129:443 -> 10.10.10.205:50501 (10.10

Interacting with a Session

You can use the **sessions -i [no.]** command to open up a specific session.

Sessions				
msf6 exploit(windows/smb/psexec_psh) > sessions -i 1				
[*] Starting interaction with 1...				
meterpreter >				

This is specifically useful when we want to run an additional module on an already exploited system with a formed, stable communication channel.

This can be done by backgrounding our current session, which is formed due to the success of the first exploit, searching for the second module we wish to run, and, if made possible by the type of module selected, selecting the session number on which the module should be run. This can be done from the second module's **show options** menu.

Usually, these modules can be found in the **post** category, referring to Post-Exploitation modules. The main archetypes of modules in this category consist of credential gatherers, local exploit suggesters, and internal network scanners.

this category consist of credential gatherers, local exploit suggesters, and internal network scanners.

Jobs

If, for example, we are running an active exploit under a specific port and need this port for a different module, we cannot simply terminate the session using `[CTRL] + [C]`. If we did that, we would see that the port would still be in use, affecting our use of the new module. So instead, we would need to use the `jobs` command to look at the currently active tasks running in the background and terminate the old ones to free up the port.

Other types of tasks inside sessions can also be converted into jobs to run in the background seamlessly, even if the session dies or disappears.

Viewing the Jobs Command Help Menu

We can view the help menu for this command, like others, by typing `jobs -h`.

Sessions
<pre>msf6 exploit(multi/handler) > jobs -h Usage: jobs [options] Active job manipulation and interaction. OPTIONS: -K Terminate all running jobs. -P Persist all running jobs on restart. -S <opt> Row search filter. -h Help banner. -i <opt> Lists detailed information about a running job. -k <opt> Terminate jobs by job ID and/or range. -l List all running jobs. -p <opt> Add persistence to job by job ID -v Print more detailed info. Use with -i and -l</pre>

Viewing the Exploit Command Help Menu

When we run an exploit, we can run it as a job by typing `exploit -j`. Per the help menu for the `exploit` command, adding `-j` to our command. Instead of just `exploit` or `run`, will "run it in the context of a job."

Sessions
<pre>msf6 exploit(multi/handler) > exploit -h Usage: exploit [options] Launches an exploitation attempt. OPTIONS: -J Force running in the foreground, even if passive. -e <opt> The payload encoder to use. If none is specified, ENCODER is used. -f Force the exploit to run regardless of the value of MinimumRank. -h Help banner. -j Run in the context of a job. <SNIP</pre>

Running an Exploit as a Background Job

Sessions

```
msf6 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 10.10.14.34:4444
```

Listing Running Jobs

To list all running jobs, we can use the `jobs -l` command. To kill a specific job, look at the index no. of the job and use the `kill [index no.]` command. Use the `jobs -K` command to kill all running jobs.

Sessions

```
msf6 exploit(multi/handler) > jobs -l
```

Jobs
====

Id	Name	Payload	Payload opts
--	----	-----	-----
0	Exploit: multi/handler	generic/shell_reverse_tcp	tcp://10.10.14.34:4444

Next up, we'll work with the extremely powerful **Meterpreter** payload.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

80ms

Terminate Pwnbox to switch location

[Start Instance](#)

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

[Cheat Sheet](#)[Download VPN
Connection File](#)**+ 1**

The target has a specific web application running that we can find by looking into the HTML source code. What is the name of that web application?

[Submit](#)**+ 1**

Find the existing exploit in MSF and use it to get a shell on the target. What is the username of the user you obtained a shell with?

[Submit](#)**+ 2**

The target system has an old version of Sudo running. Find the relevant exploit and get root access to the target system. Find the flag.txt file and submit the contents of it as the answer.

[Submit](#)[← Previous](#)[Next →](#)[Cheat Sheet](#)[Go to Questions](#)[Table of Contents](#)

Introduction

Preface



Introduction to Metasploit



Introduction to MSFconsole



MSF Components

 Modules



Targets



 Payloads



Encoders



Databases



Plugins & Mixins



MSF Sessions

 Sessions & Jobs

 Meterpreter

Additional Features

Writing & Importing Modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left